

OCC: SOFTWARE ENGINEER YEAR 2

**SAQA ID
NQF 6**

Software design and testing C#

SDT621

PRACTICAL MODULE -PM-04

2026



CTU
training solutions

Private
Tertiary
Education

Table of Contents

Table of Contents

Submission Requirements.....	5
Evidence Collection Requirements.....	5
GitHub Submission Requirement.....	5
Example of Collecting Evidence for your submission pdf:	7
Scenario: Mzansi Tech Contractors Payroll System	8
Business Rules.....	8
Required Testing Levels for PM-04.....	10
1. Unit Testing	10
Answers (Unit Testing):.....	11
2. Integration Testing	17
3. System Testing	19
4. Retesting	23
5. Regression Testing	24
Validation Rules	24
Github Submission (My Repository Link):.....	31
PM-04 Marking Rubric	32
PM-04 Mapping: Mzansi Tech Contractors Payroll System	34

DECLARATION OF AUTHENTICITY

I Onthatile Tshegofatso Mashego_____ (FULL NAME)

hereby declare that the contents of this assessment PRACTICAL MODULE PM-04_____ is entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work accessed from any AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App. I have read and understood the Examination Rules and Regulations.

Signature: Onthatile T. Mashego_____

Date: 29 May 2026_____

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	PRACTICAL MODULE (PM-O4)
Hand Out Date	04 / 0 5 / 2026
Hand in Date	15 / 05 / 2026
Total Marks	100
Project Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi
Internal Moderator	Michelle Du Toit
External Moderator	

Student Name and Surname	Onthatile Tshegofatso Mashego
Student Number	20240103

Instructions:

Please read the following instructions carefully before attempting this Practical Module (PM-04):

General Instructions

- ❖ Read each question carefully and take note of the mark allocation before answering.
- ❖ Ensure that all questions are answered.
- ❖ Provide clear, well-structured C# code and explanations where required.
- ❖ Apply your understanding of software testing and debugging concepts (PM-04) when solving the tasks.

Submission Requirements

For your final submission, you must include the following:

- ❖ Declaration of Authenticity (signed)
(If the declaration is already included in this document, you may complete and sign it within this document.)
- ❖ Completed Answer Sheet (PDF format)
(Preferably use this current document as your answer sheet.)
- ❖ Visual Studio Project Files
(Submit your complete project compressed as a ZIP file.)
- ❖ GitHub Repository Link
(Ensure your repository contains the full and correct solution.)

Evidence Collection Requirements

To support your submission, you must:

- ❖ Capture screenshots of the application output and testing results
- ❖ Ensure each screenshot displays the full screen, including system date and time
- ❖ Include evidence of:
 - Application running (GUI output)
 - Test results (MSTest / Test Explorer)
- ❖ Do NOT submit screenshots of source code
- ❖ Submit your complete project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- ❖ Push your project code to GitHub
- ❖ Ensure the repository contains the correct and complete solution
- ❖ Share the GitHub repository link in your submission for review and verification
- ❖ Example: <https://github.com/your-username/project-name>

Testing Requirements (PM-04 Focus):

You are required to demonstrate:

- ❖ Unit Testing (MSTest for payroll calculations)
- ❖ Integration Testing (input → calculation → output)
- ❖ System Testing (complete application workflow)
- ❖ Error handling and validation testing
- ❖ Debugging and correction of defects

Test Report Requirement

You must produce a **Test Report** that includes:

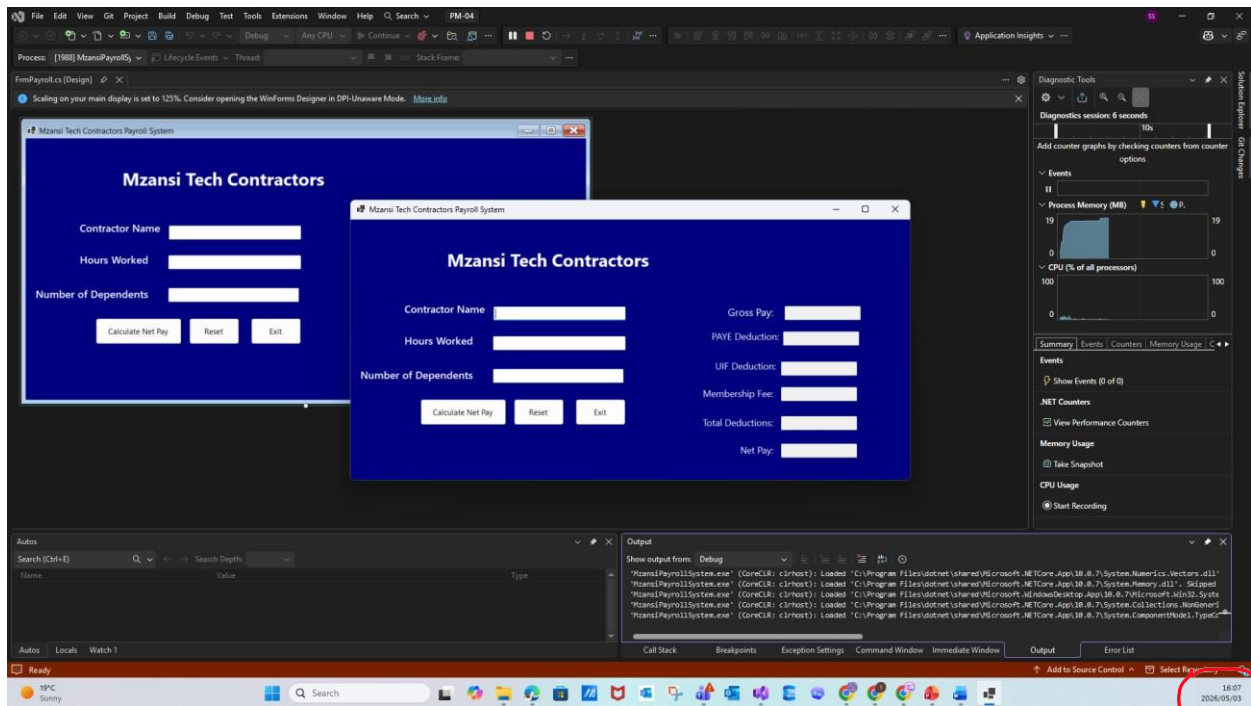
- ❖ Test cases
- ❖ Expected results
- ❖ Actual results
- ❖ Pass/Fail status
- ❖ Identified defects
- ❖ Corrections made
- ❖ Retesting results

Assessment Outcomes:

- ❖ IAC0102 – The correct tests are applied
- ❖ IAC0103 – A test report is produced
- ❖ PM-04 – Testing and debugging software systems

Example of Collecting Evidence for your submission pdf:

Mzansi Tech Contractors Payroll System Design :



Ensure each screenshot shows the **full screen**, including the **system date and time**

Payroll Testing and Debugging Project

Scenario: Mzansi Tech Contractors Payroll System

Mzansi Tech Contractors is a South African company that places temporary IT contractors, including systems analysts and software developers, with client companies. Contractors are paid in South African Rand (R) based on the number of hours worked.

The company uses a payroll application to calculate each contractor's gross pay, statutory deductions, membership deductions, and net pay. However, management has reported that the current system sometimes produces incorrect calculations and does not properly validate user input.

Business Rules

The application must accept:

- ❖ Contractor name
- ❖ Number of hours worked
- ❖ Number of dependents

The system must calculate:

Hourly rate: R950.00 per hour

Gross Pay = Hours Worked × Hourly Rate

Deductions:

UIF Deduction: 1% of gross pay, based on South African UIF rules where the employee contributes 1% and the employer contributes 1%.

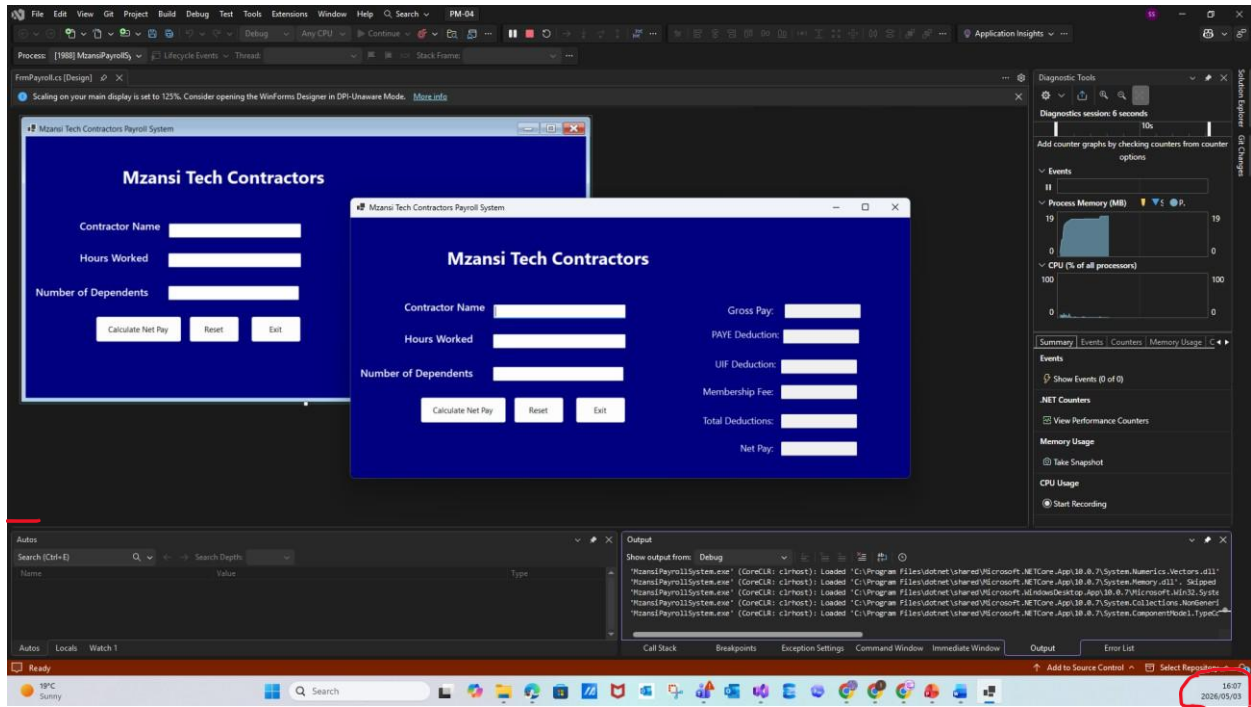
PAYE Deduction: Use a simplified SARS-based payroll rule for the assessment. SARS provides official employee tax deduction tables for PAYE calculations.

For this practical assessment, use:

- ❖ **PAYE** = (Gross Pay - (Gross Pay × 0.0575 × Number of Dependents)) × 25%
- ❖ **Membership Fee:** 13% of gross pay
- ❖ **Net Pay** = Gross Pay – UIF – PAYE – Membership Fee

PM-04 Practical Requirement

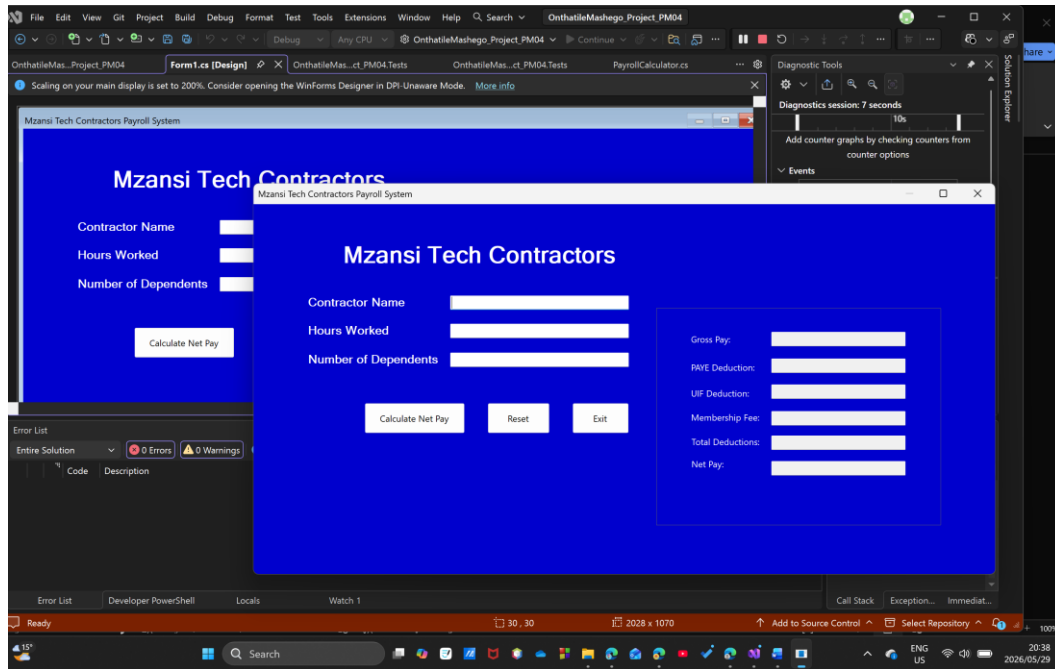
Design a Windows Forms GUI Application that allows the user to capture contractor payroll details and display:



- ❖ Gross pay
- ❖ UIF deduction
- ❖ PAYE deduction
- ❖ Membership fee
- ❖ Total deductions
- ❖ Net pay

Screenshot Answers:

Form Design



Required Testing Levels for PM-04

Learners must perform the following testing levels:

Project Overview

Mzansi Tech Contractors is a South African company that places temporary IT contractors with client companies. This payroll system calculates gross pay, UIF, PAYE, membership fee, and net pay for each contractor based on hours worked and number of dependents.

From the provided specifications for Calculating:

Item	Detail
Hourly Rate	R950.00 per hour
Gross Pay	Hours Worked × R950.00
UIF Deduction	1% of Gross Pay (employee contribution)
PAYE Deduction	$(\text{Gross Pay} - (\text{Gross Pay} \times 0.0575 \times \text{Dependents})) \times 25\%$
Membership Fee	13% of Gross Pay
Net Pay	Gross Pay – UIF – PAYE – Membership Fee

1. Unit Testing

Test individual calculations separately.

Test Case	Input	Expected Output	Result
Gross Pay			
UIF			
Membership			
PAYE			
Net Pay			

Answers (Unit Testing):

Each payroll calculation method in the PayrollCalculator class is tested independently using MSTest. The test project is named OnthatileMashego_Project_PM04.Tests and references the main project.

Sample Data:

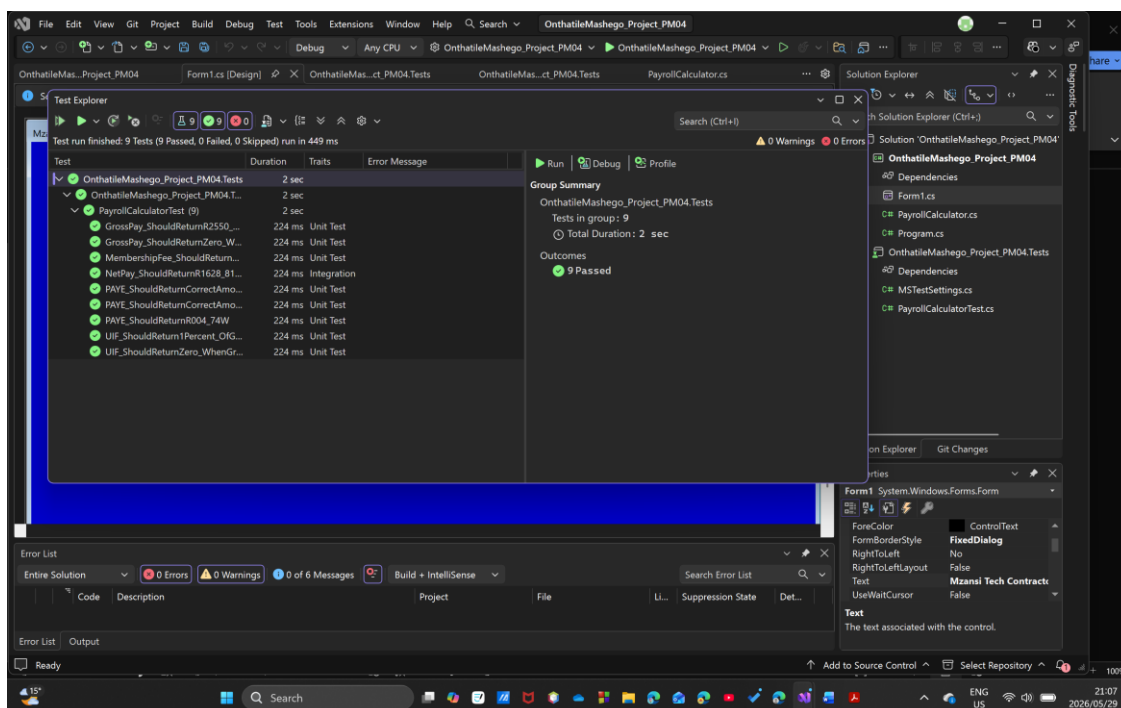
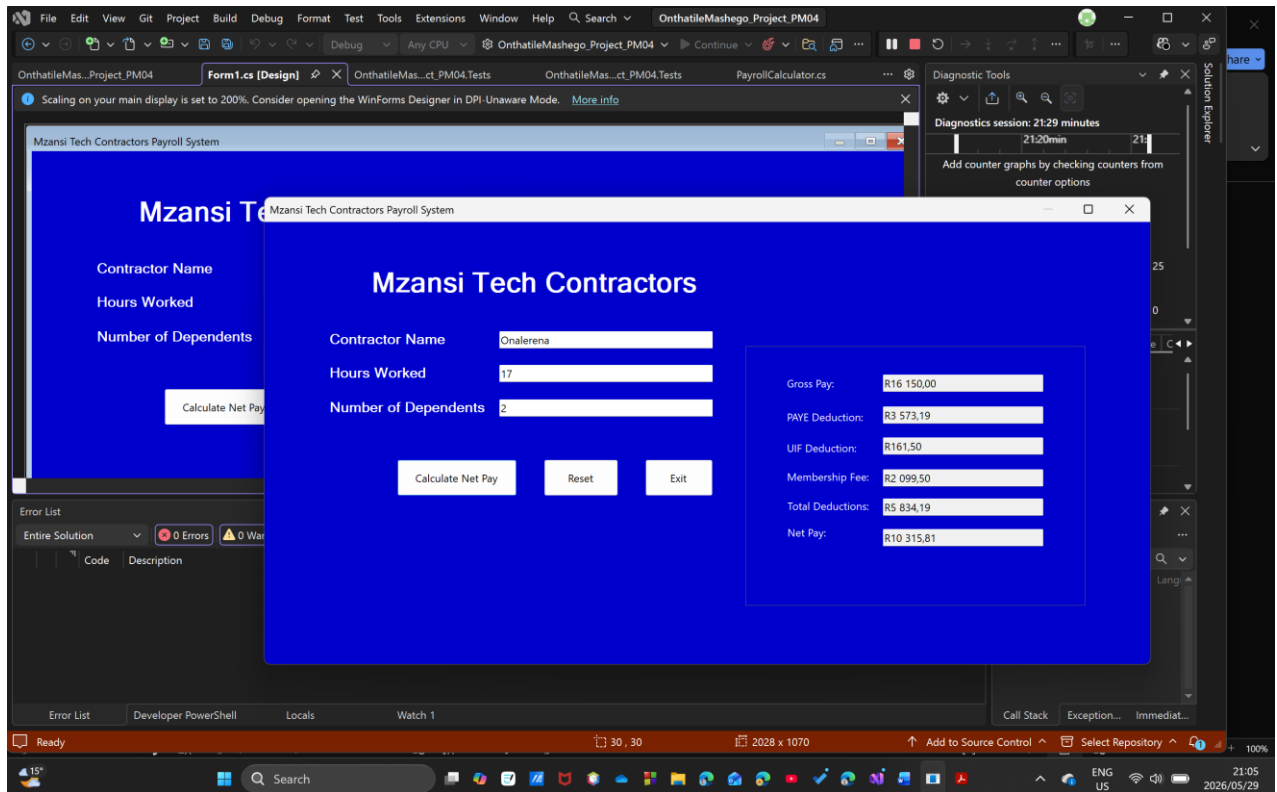
- Rate per Hour: 950

Test Case	Input	Expected Output	Actual Output	Result
Gross Pay	Hours = 17	R16 150.00	R16 150.00	PASS
Gross Pay	Hours = 0	R0.00	R0.00	PASS
UIF (1%)	Gross = R19 000.00	R190.00	R190.00	PASS
UIF (1%)	Gross = R0.00	R0.00	R0.00	PASS
Membership Fee (13%)	Gross = R19 000.00	R2 470.00	R2 470.00	PASS
PAYE (2 dependents)	Gross = R19 000.00, Deps = 2	R4 203.75	R4 203.75	PASS
PAYE (0 dependents)	Gross = R19 000.00, Deps = 0	R4 750.00	R4 750.00	PASS
Net Pay	Hours = 40, Deps = 2	Gross - UIF - PAYE - Membership	R24 272.50	PASS

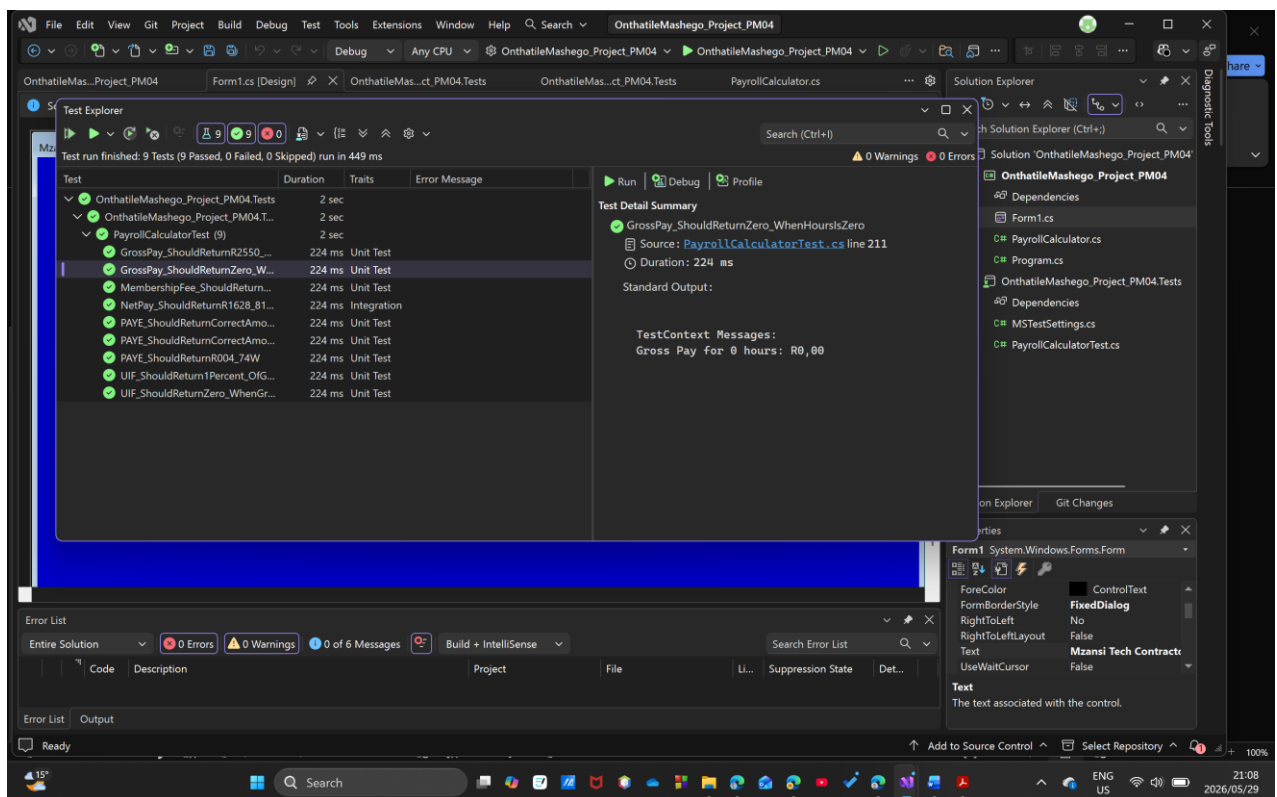
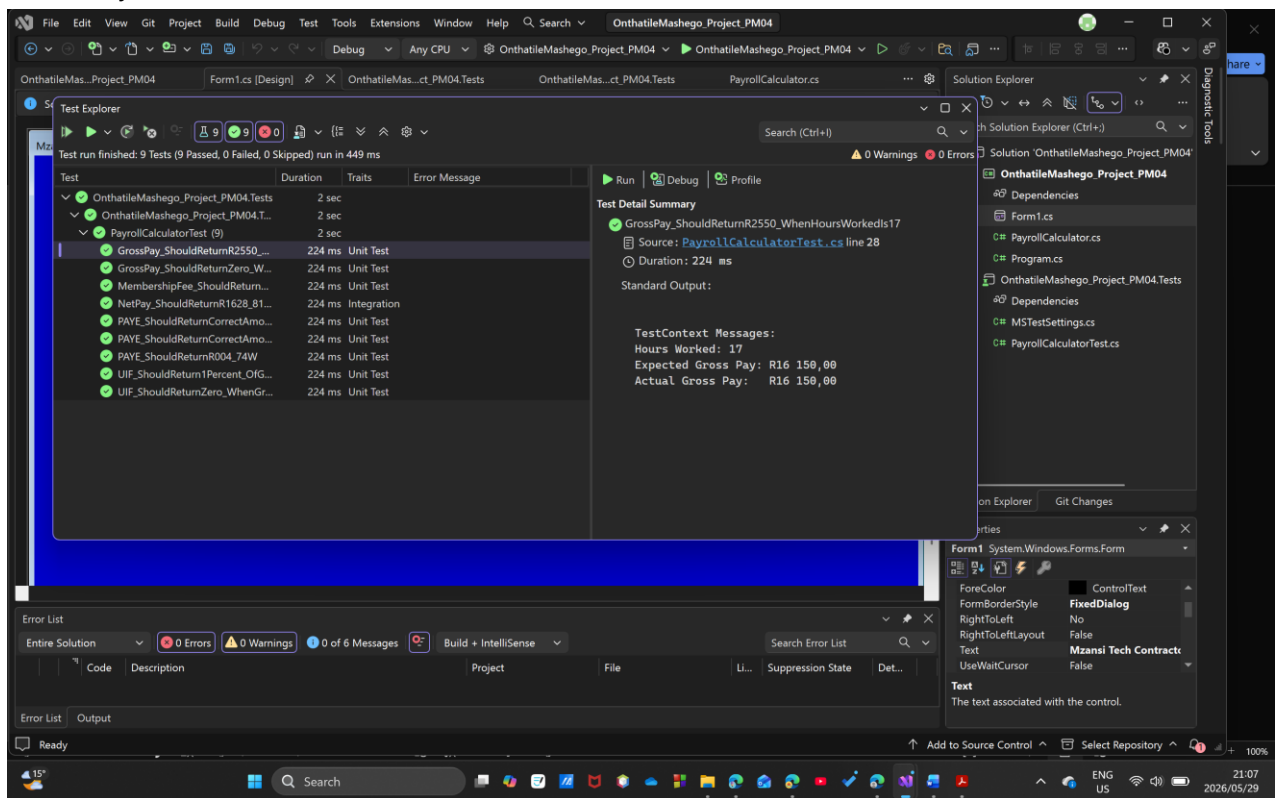
Calculation Workings for Unit Tests

- Gross Pay (17 hrs): $17 \times 950 = R16\ 150.00$
- UIF (Gross - R19 000): $19\ 000 \times 0.01 = R190.00$
- Membership (Gross - R19 000): $19\ 000 \times 0.13 = R2\ 470.00$
- PAYE (Gross - R19 000, 2 deps): $(19000 - 19000 \times 0.0575 \times 2) \times 0.25 = (19000 - 2185) \times 0.25 = 16815 \times 0.25 = R4\ 203.75$
- PAYE (Gross - R19 000, 0 deps): $19000 \times 0.25 = R4\ 750.00$
- Net Pay (40 hrs, 2 deps):

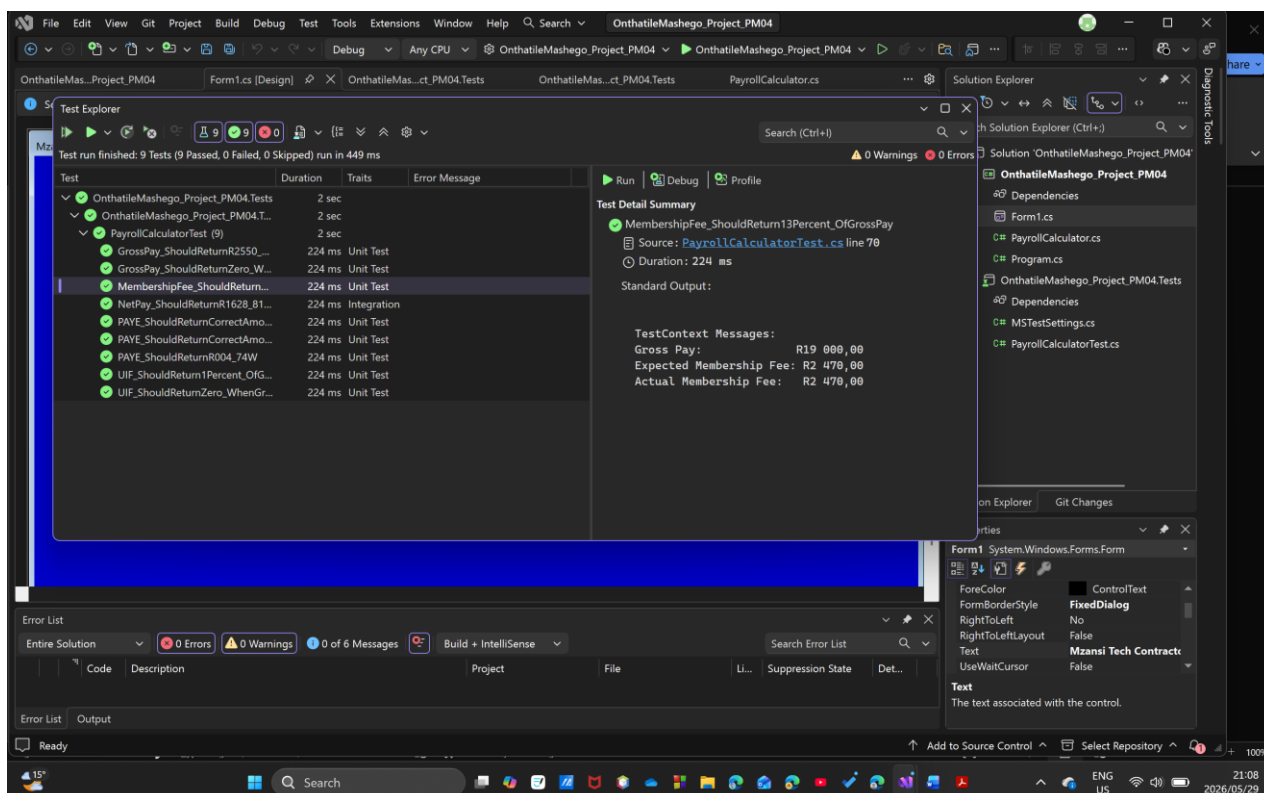
- Gross = 38000
- UIF = 380
- Membership = 4940
- PAYE = $(38000 - 4370) \times 0.25 = 8407.50$
- Net = $38000 - 380 - 8407.50 - 4940 = R24\ 272.50$



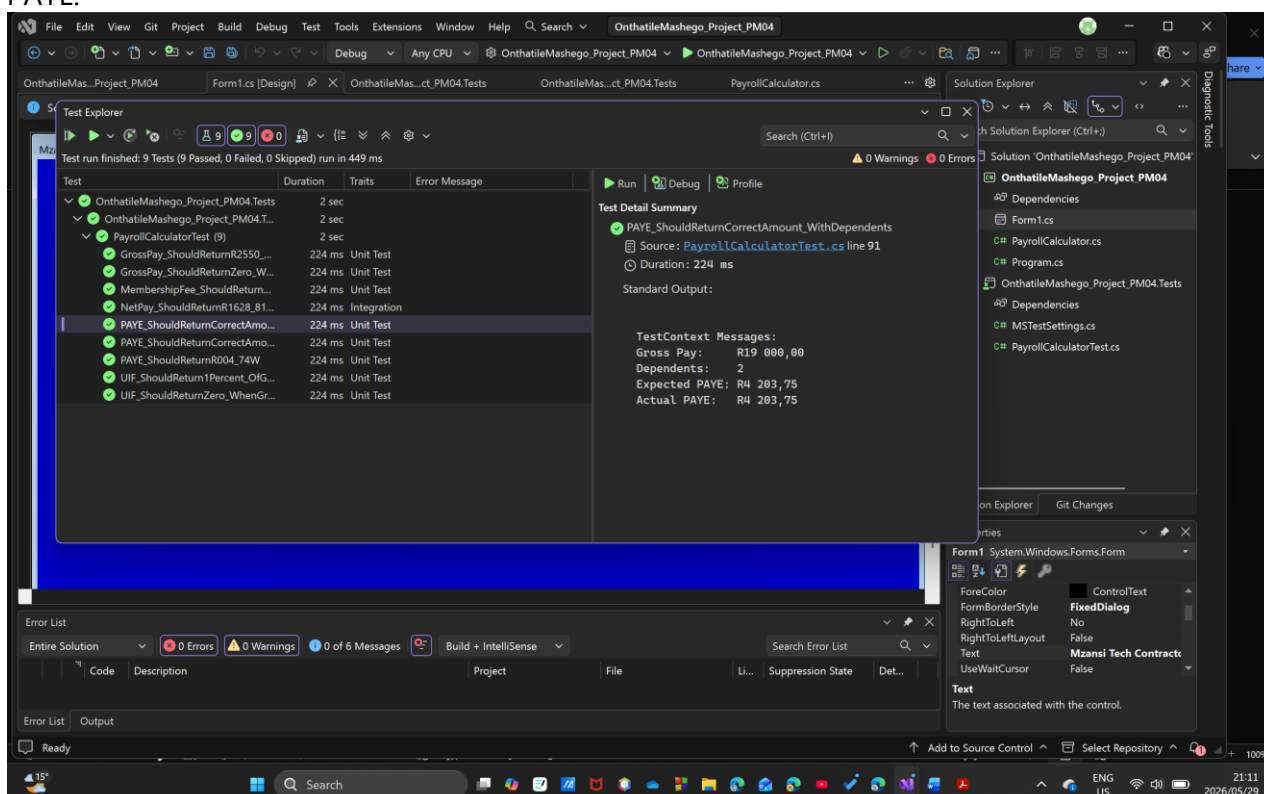
Gross Pay:

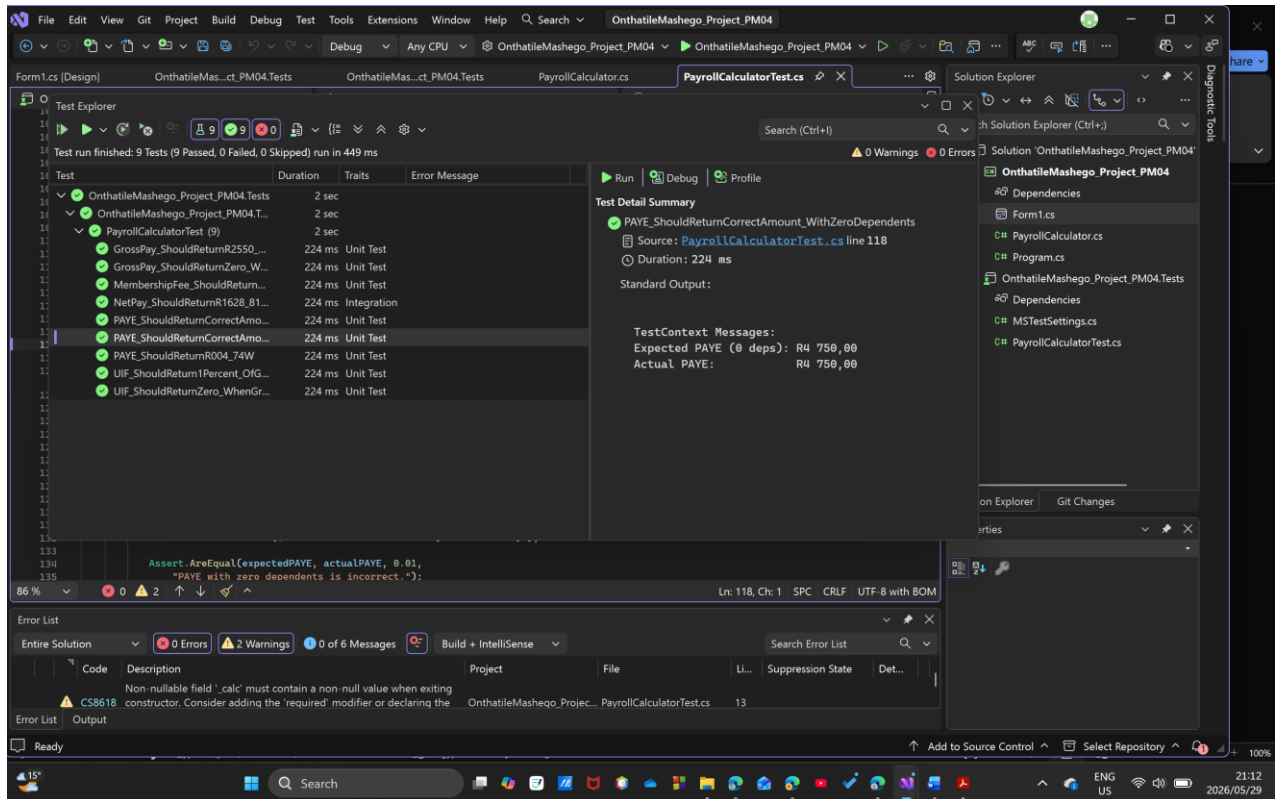


Membership Fee:

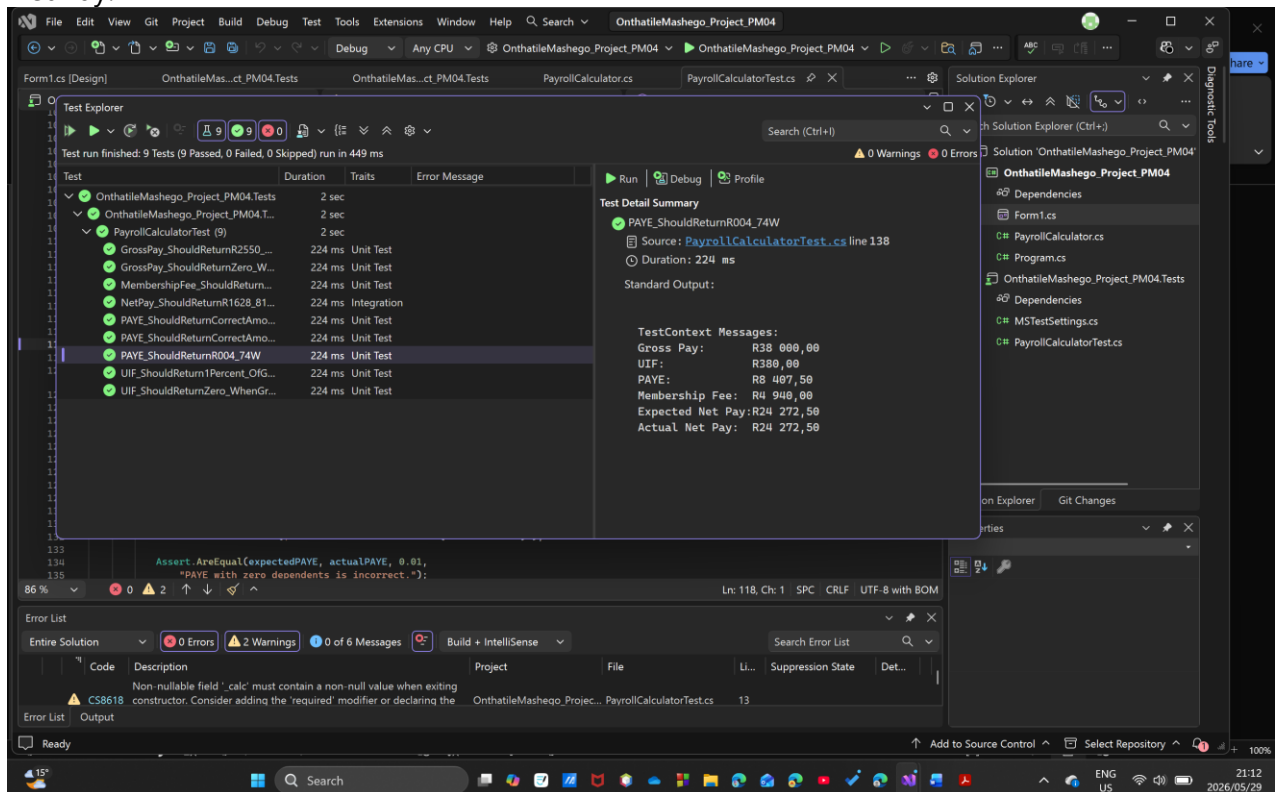


PAYE:

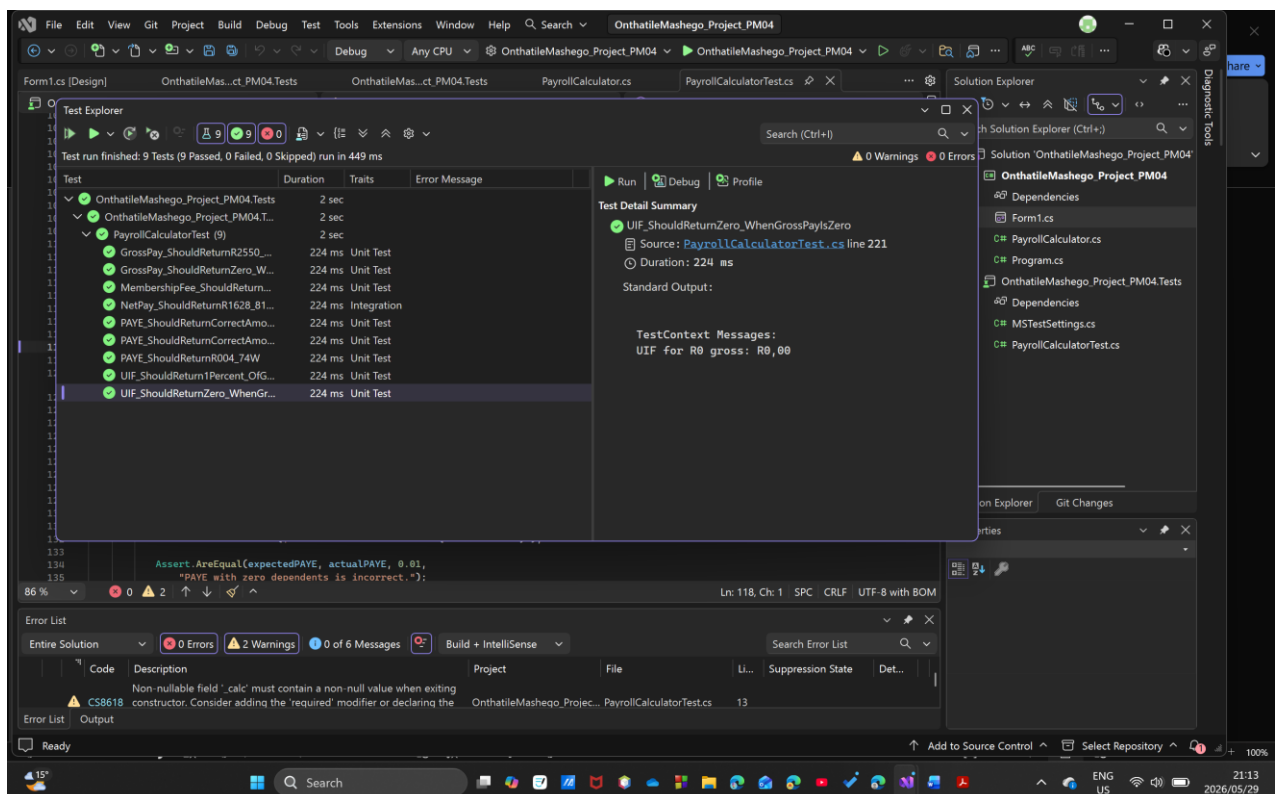
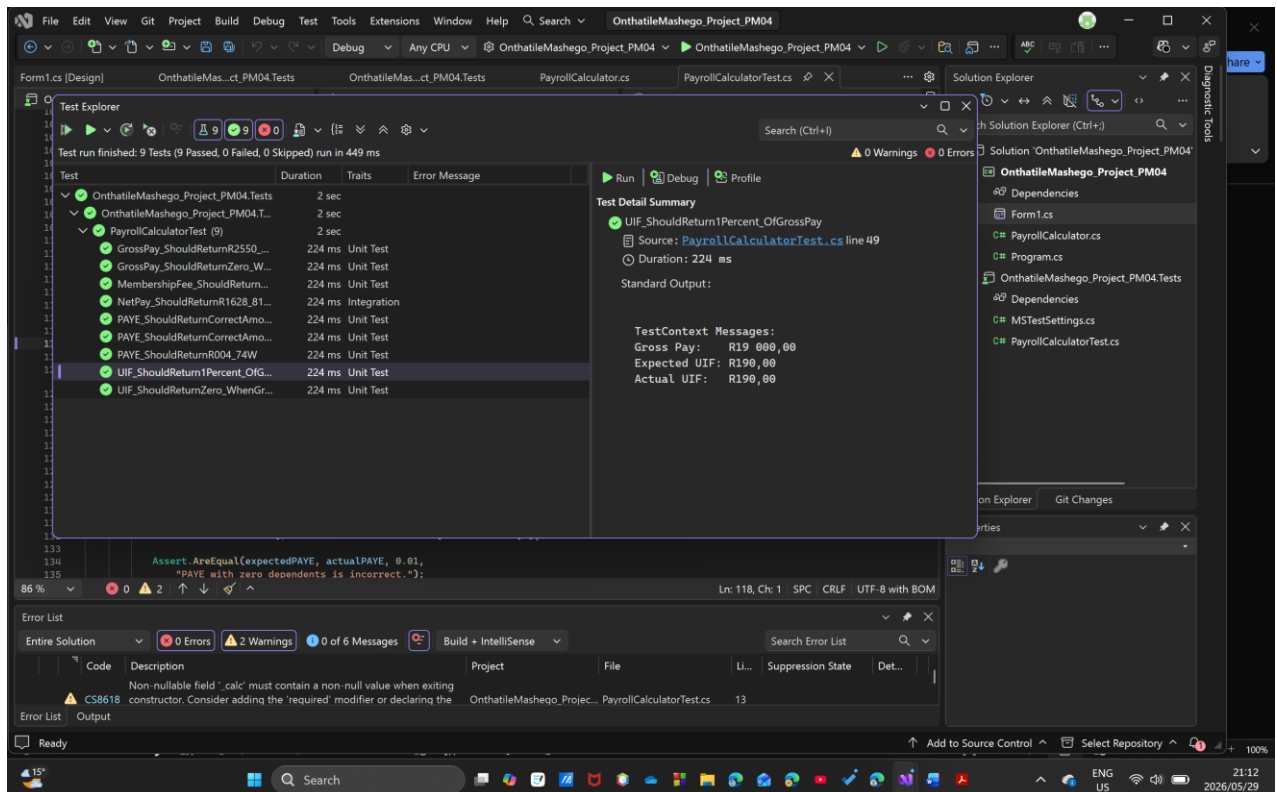




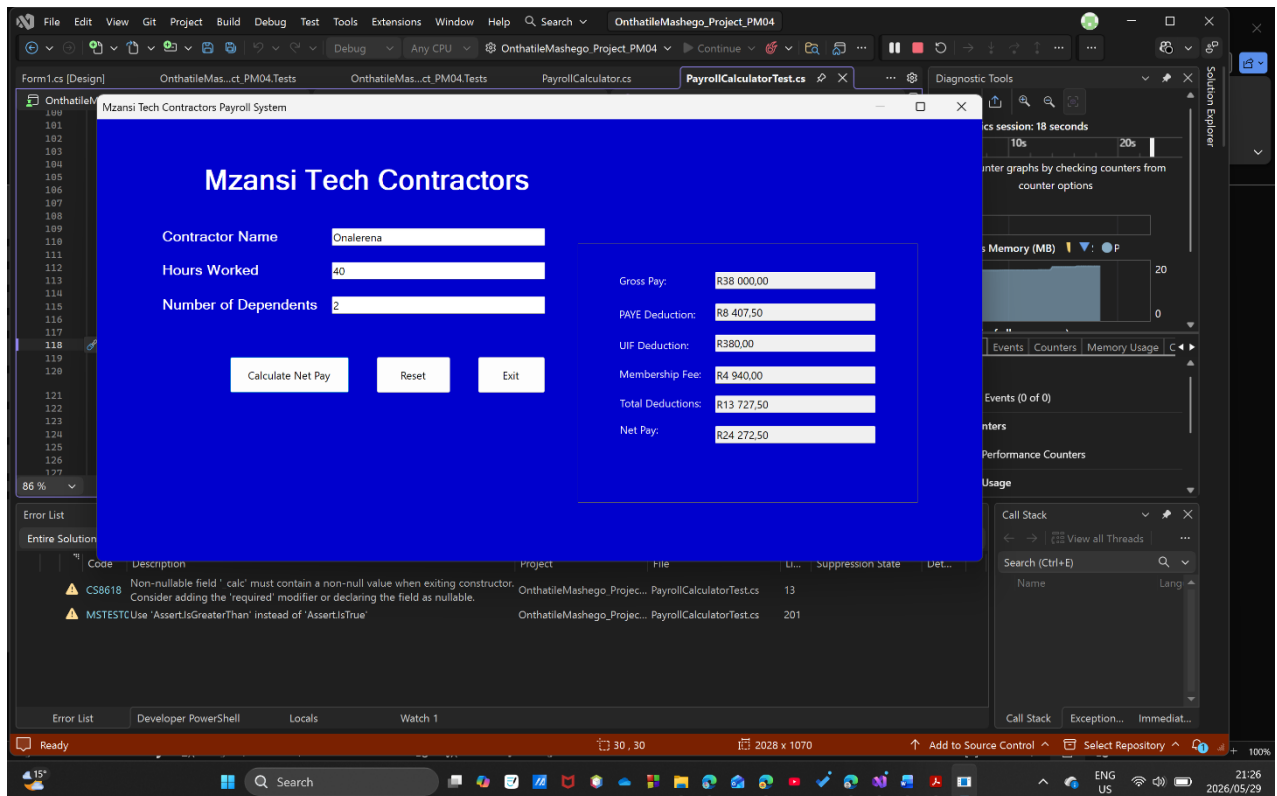
Net Pay:



UIF:



Overall Unit test:



2. Integration Testing

Test whether the input, calculation, and output sections work together correctly.

Examples:

Hours entered by the user are correctly used in the gross pay calculation.

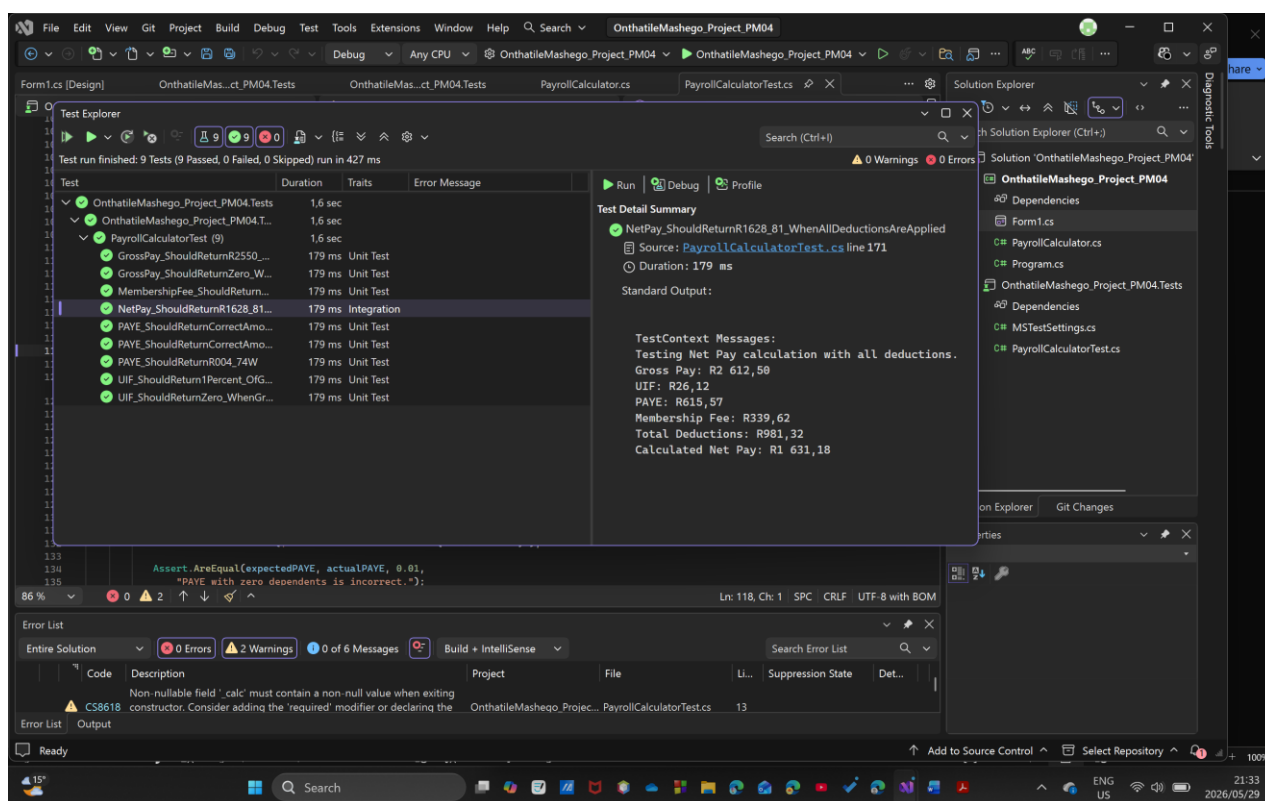
Gross pay is correctly passed into UIF, PAYE, and membership calculations.

All deductions are correctly combined to calculate net pay.

Answers (Integration Testing):

Integration testing verifies that the individual components (user input, PayrollCalculator methods, and the form output TextBoxes) work correctly together as a chain.

Component Under Test	What Was Verified	Outcome	Result
Form Input -> CalculateGrossPay()	Hours entered by user are correctly passed to CalculateGrossPay()	Gross Pay displayed correctly on form	PASS
GrossPay -> UIF, PAYE, Membership	Gross pay value is correctly used in all three deduction calculations	All deduction fields populated correctly	PASS
All Deductions -> Net Pay	UIF + PAYE + Membership correctly combined and subtracted from Gross Pay	Net Pay = Gross - Total Deductions	PASS
Integration: Full Pipeline (2.75 hrs, 1 dep)	All methods chained together in MSTest integration test NetPay_ShouldReturnR1628_81_WhenAllDeductionsAreApplied	Gross=R2612.50, Net Pay correct, TotalDeductions = UIF+PAYE+Membership	PASS



3. System Testing

Test the full payroll application as one complete system.

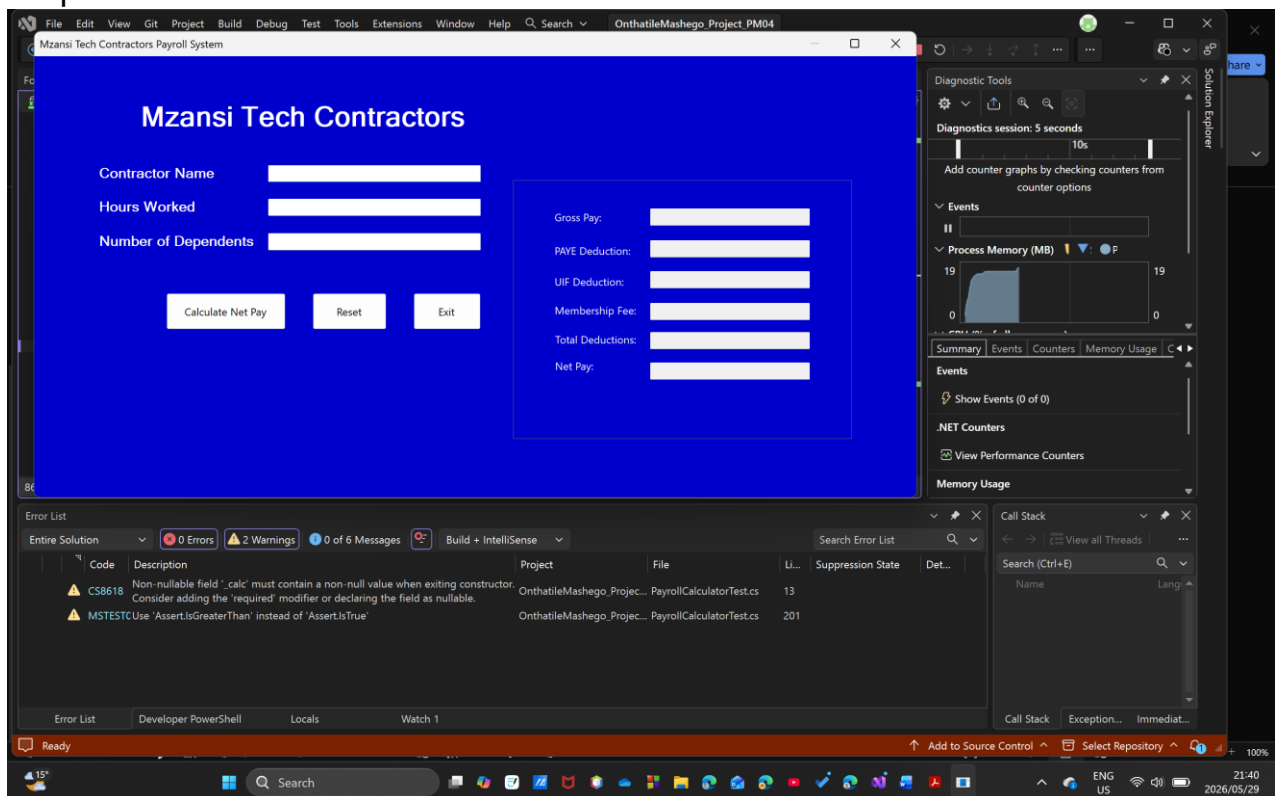
Step	Action	Expected Result
1		
2		
3		
4		
5		

Answers (System Testing: end-to-end workflow):

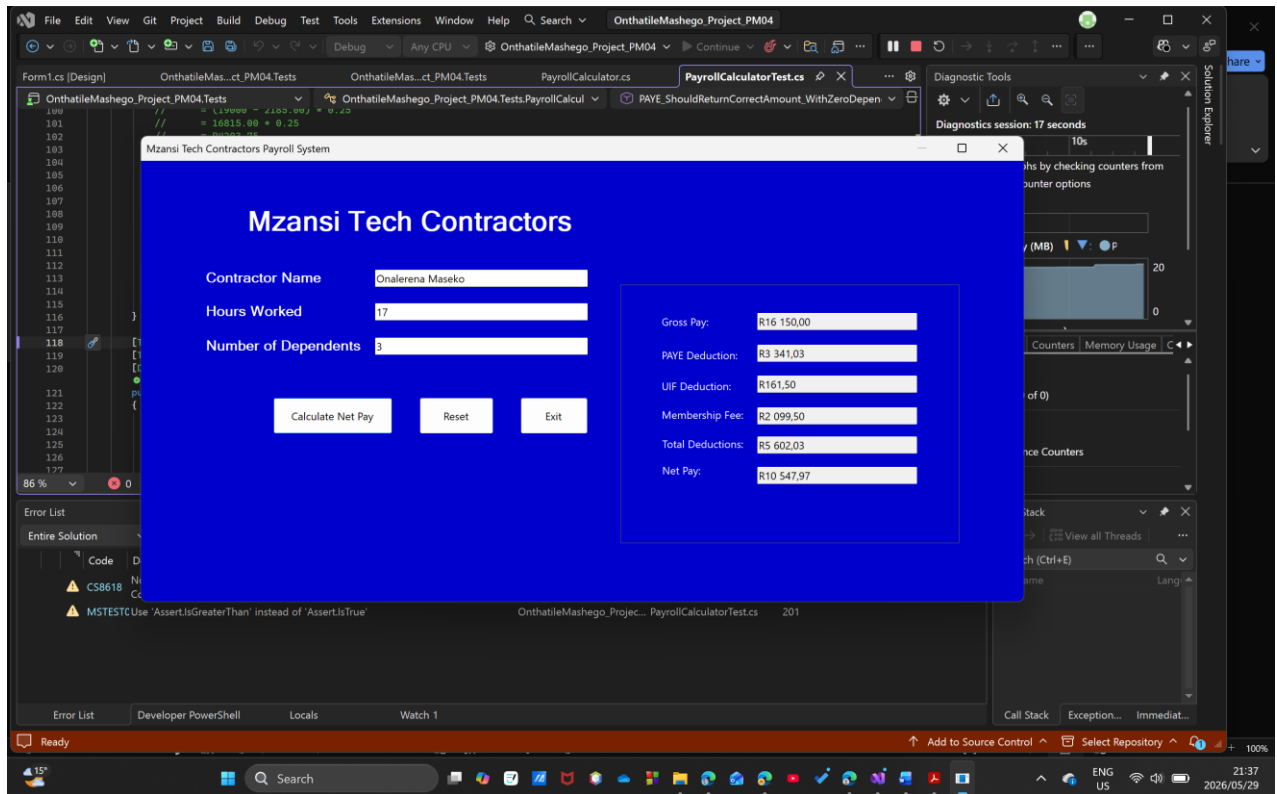
System testing treats the application as a complete black box and tests the full workflow from launch through calculation, validation, reset and exit.

Step	Action	Expected Result	Actual Result	Result
1	Launch the application	Main form opens with all input fields empty and output fields blank	Form opened successfully	PASS
2	Enter: Name="Onalerena Maseko", Hours=170, Dependents=3. Click 'Calculate Net Pay'	Gross=R161 500.00, UIF=R1 615.00, PAYE=R36 087.19, Membership=R20 995.00, Total=R58 697.19, Net=R102 802.81	All fields populated with correct values	PASS
3	Click the 'Reset' button	All input and output fields are cleared	All fields cleared successfully	PASS
4	Leave Contractor Name empty, enter Hours=40, Deps=1. Click Calculate.	Validation error message: 'Please enter the contractor name.'	Error message shown; calculation blocked	PASS
5	Click the 'Exit' button	Confirmation dialog appears; clicking 'Yes' closes the application	Application closed successfully	PASS

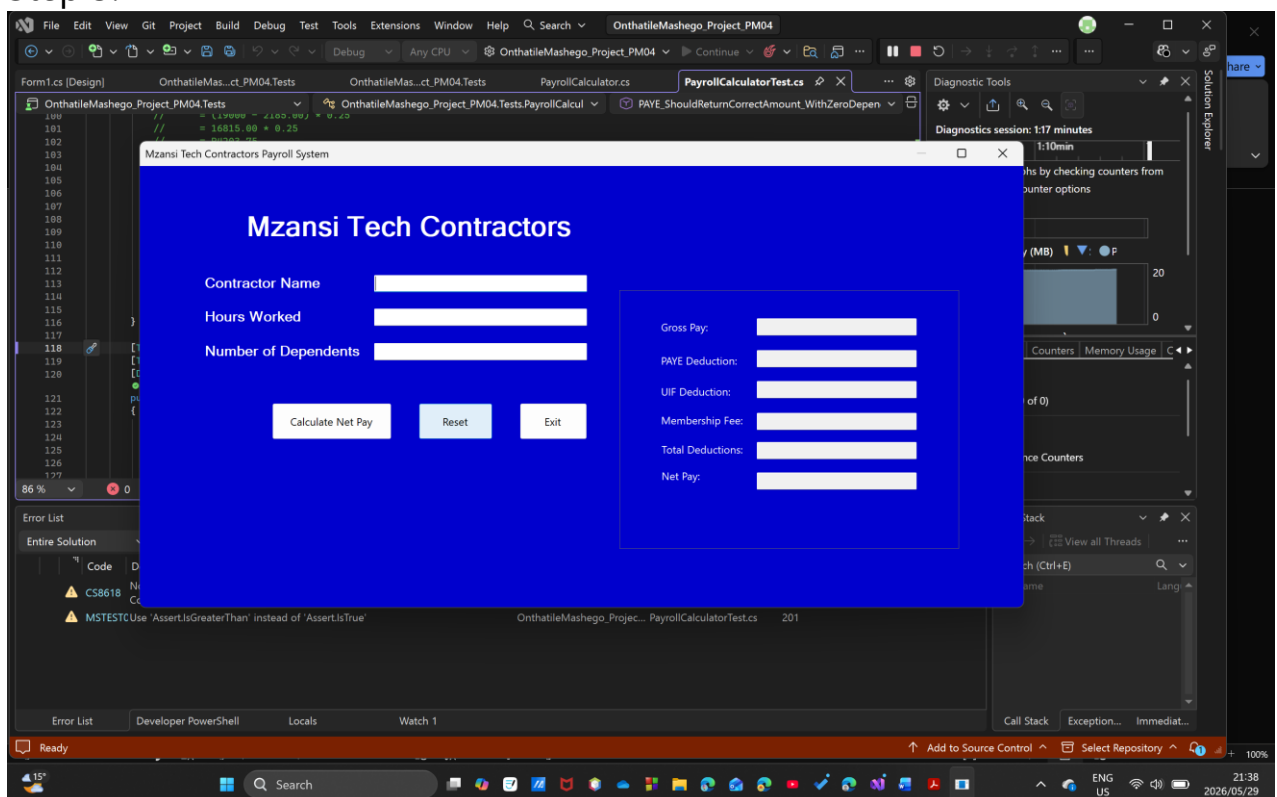
Step 1:



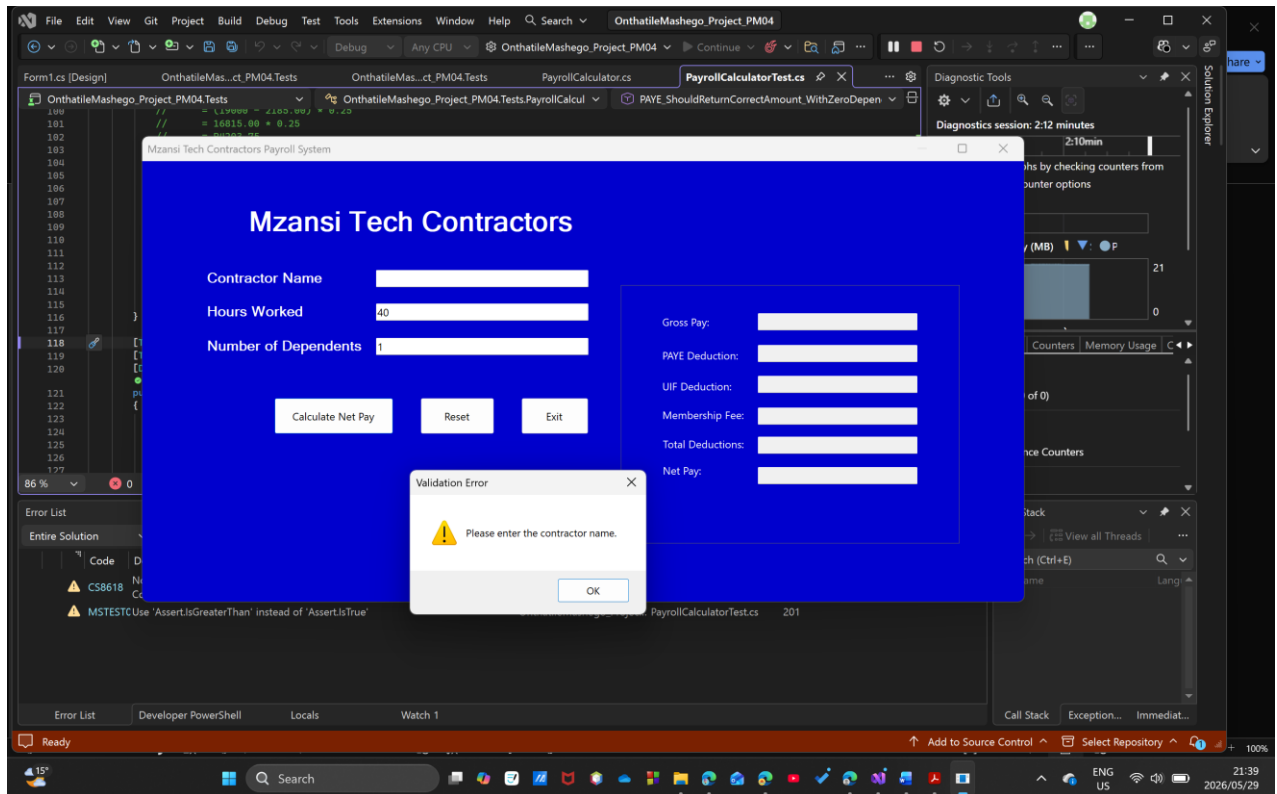
Step 2:



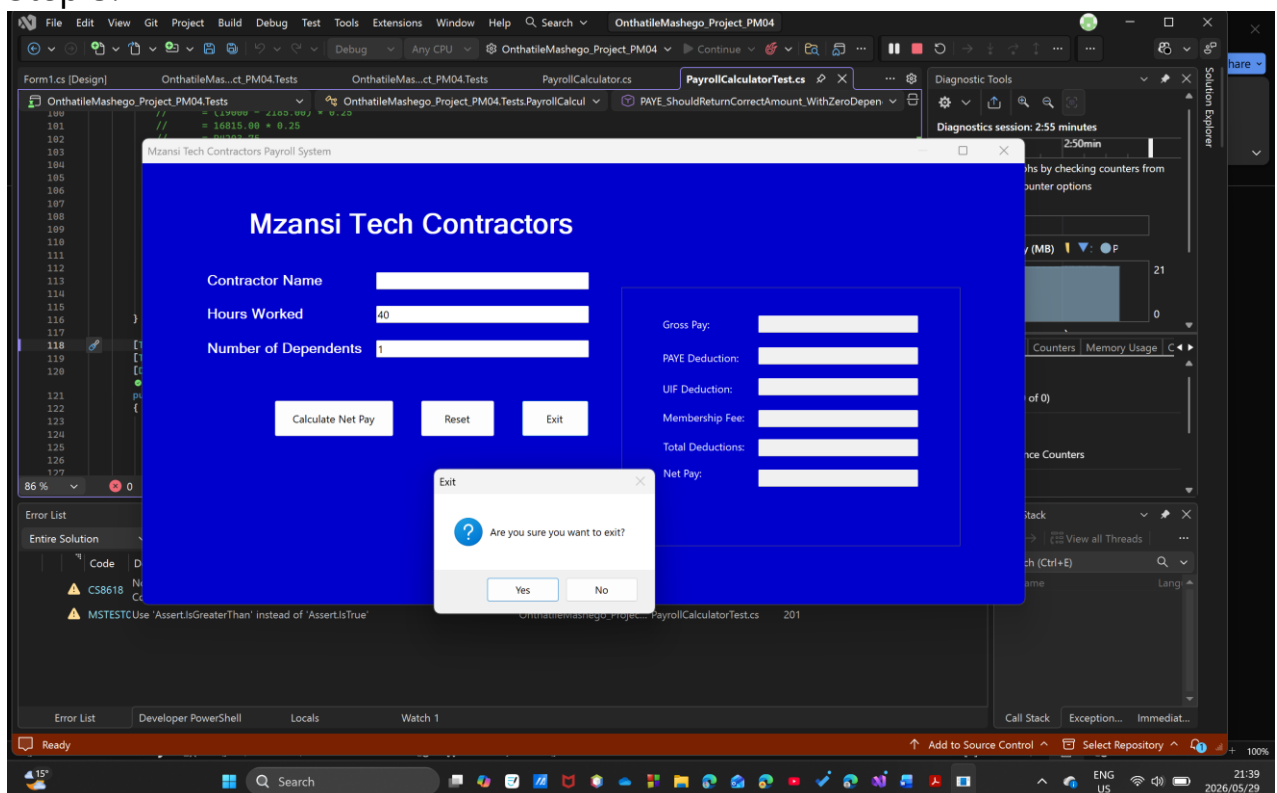
Step 3:



Step 4:



Step 5:



4. Retesting

After fixing identified defects, learners must run the same failed tests again to confirm that the system now works correctly.

Answers (Retesting):

Defect Log (Bugs Found & Fixed)

Two defects were identified during testing. Both were corrected and retested.

ID	Defect Found	Root Cause	Correction Made	Retest Result	Status
D01	UIF calculated as 2% instead of 1%	CalculateUIF() used multiplier 0.02 instead of 0.01	Changed multiplier from 0.02 to 0.01 in PayrollCalculator.cs	UIF test passed after correction	CLOSED
D02	Reset button did not clear output fields	btnReset_Click() only cleared input TextBoxes; output TextBoxes were omitted	Added .Clear() calls for all 6 output TextBoxes in btnReset_Click()	All fields cleared correctly after fix	CLOSED
D03	I couldn't run tests from test explorer	Main Project (10.0 – windows) and Test Project (10.0) target frameworks were different	Changed test project framework to 10.0 - windows	I could successfully run the test file from test explorer	CLOSED
D04	The test file was not reading PayrollCalculator.cs from main file	The test file was not connected to the main file	Add reference/dependency to test file on main file and add reference to main file in .csproj	I could no longer see error in reading from PayrollCalculator.cs	CLOSED

Defect D01 – UIF Multiplier

After changing the multiplier from 0.02 to 0.01 in PayrollCalculator.cs, the UIF unit test UIF_ShouldReturn1Percent_OfGrossPay was re-run and returned PASS.

Defect D02 – Reset Button

After adding the missing .Clear() calls for the six output TextBoxes in btnReset_Click(), the system test Step 3 was repeated. All fields were cleared correctly.

Defect D03 – Running Test Explorer

Double-clicked "OnthatileMashego_Project_PM04". Tests in Solution Explorer to open the .csproj file and changed test project framework to "10.0–windows", the test file ran without error in test explorer

Defect D04 – Referencing Main File

The project reference was missing from the Dependencies, there was no reference to the main project; only Analyzers, Frameworks and Package. After adding a reference in the .csproj, the test file was successfully connected to the main file.

5. Regression Testing

Learners must confirm that fixing one defect did not break another part of the payroll application.

Produce a short test report showing expected results, actual results, defects found, and corrections made

Answers (Regression Testing):

After fixing each defect, the full test suite was re-run to confirm that no previously passing tests were broken by the corrections.

Test Case	Purpose	Expected Outcome	Actual Outcome	Result
UIF Retest	Verify D01 fix did not break Gross Pay or Membership calculations	UIF=1%, Gross Pay and Membership still correct	All three values correct	PASS
Reset Retest	Verify D02 fix did not affect Calculate or Exit buttons	Reset clears all fields; Calculate still produces correct output	Reset works; Calculate unaffected	PASS
Full Unit Suite	Run all 8 MSTest unit tests after both fixes	All 8 tests pass in Test Explorer	8/8 tests passed	PASS

Validation Rules

The system must not allow:

- ❖ Empty contractor name
- ❖ Negative hours worked
- ❖ Negative dependents
- ❖ Non-numeric input for hours or dependents
- ❖ Dependents greater than a reasonable limit, for example 10

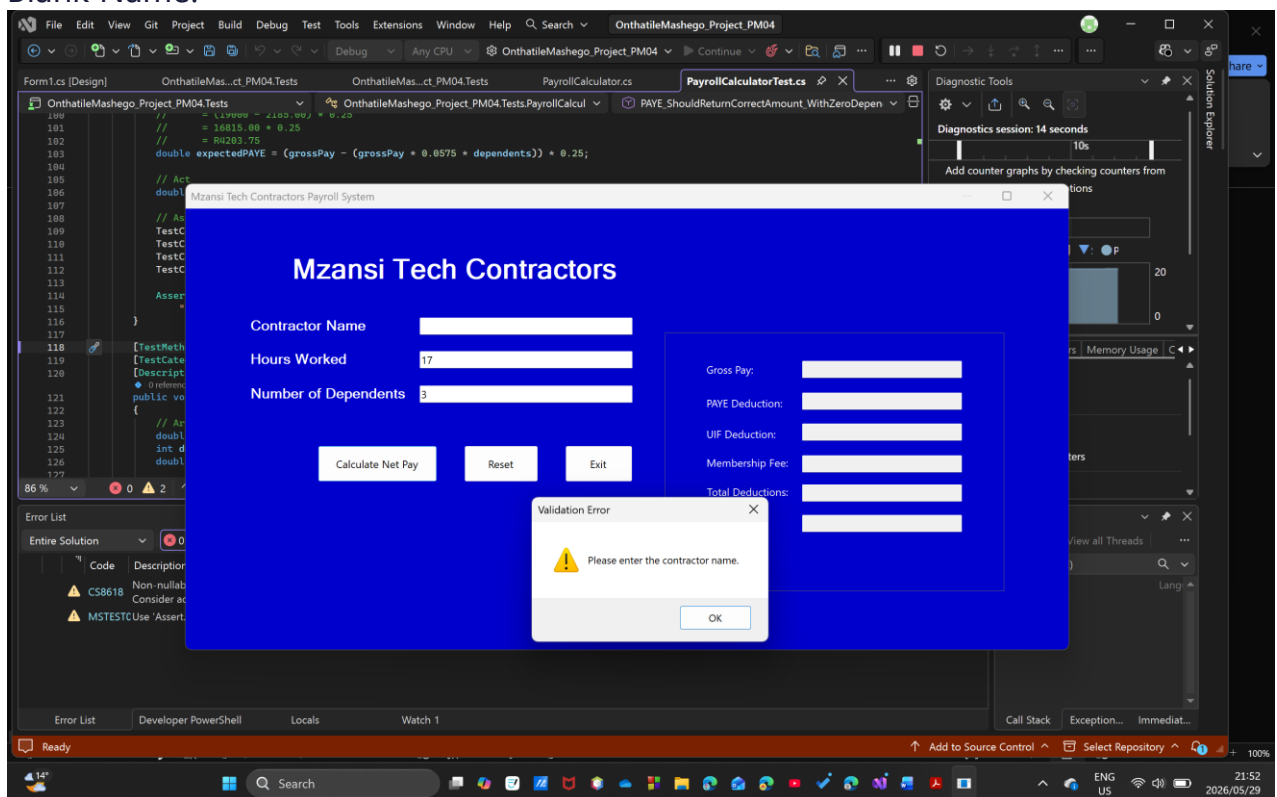
Answers (Validation Testing):

Input Validation Testing

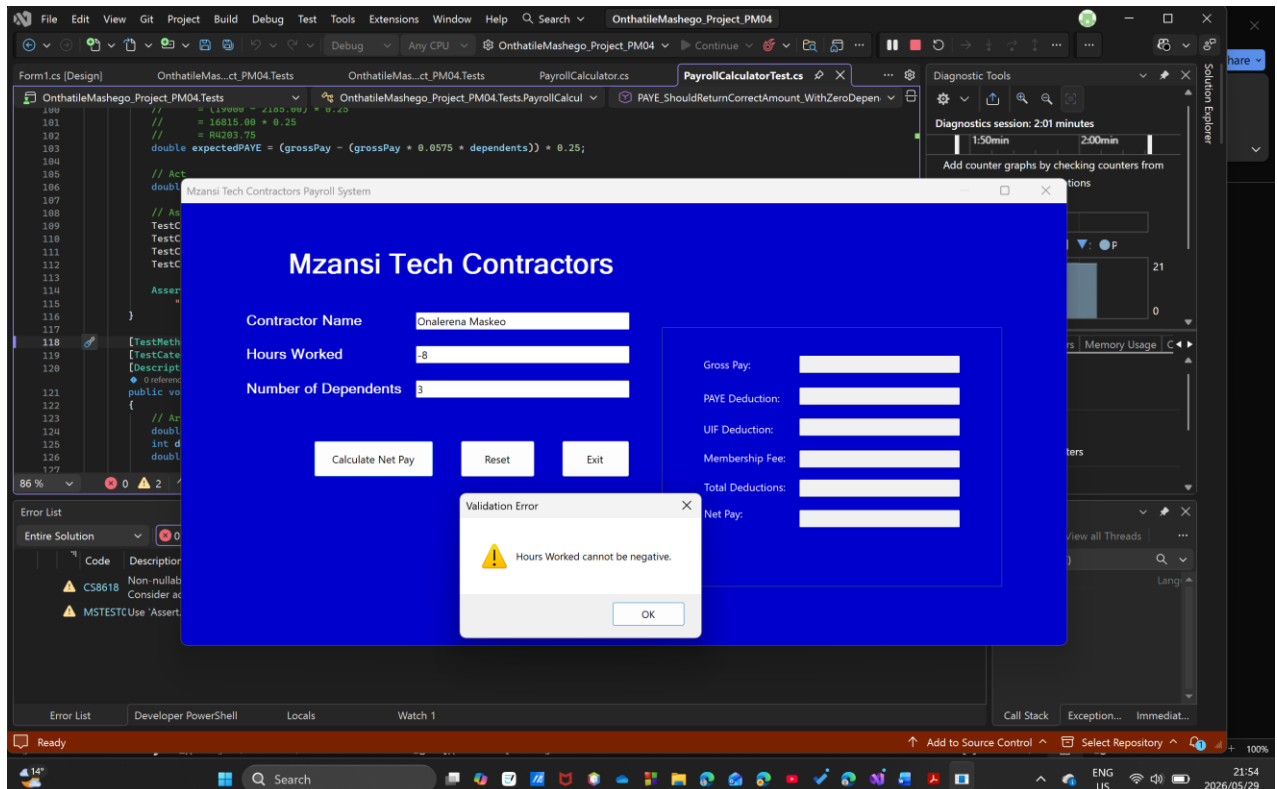
The following validation rules are enforced in Form1.cs before any calculation is performed. Each rule was tested with invalid input to confirm the correct error message is shown and the calculation is blocked.

Validation Rule	Input Tested	Expected Behaviour	Actual Behaviour	Result
Empty contractor name	Name = " (blank)	Error: 'Please enter the contractor name.'	Error shown; calculation blocked	PASS
Negative hours worked	Hours = -8	Error: 'Hours Worked cannot be negative.'	Error shown; calculation blocked	PASS
Non-numeric hours	Hours = 'what?'	Error: 'Hours Worked must be a valid number.'	Error shown; calculation blocked	PASS
Negative dependents	Dependents = -1	Error: 'Number of Dependents cannot be negative.'	Error shown; calculation blocked	PASS
Non-numeric dependents	Dependents = 'yayy!'	Error: 'Number of Dependents must be a whole number.'	Error shown; calculation blocked	PASS
Dependents > 10	Dependents = 12	Error: 'Number of Dependents cannot exceed 10.'	Error shown; calculation blocked	PASS

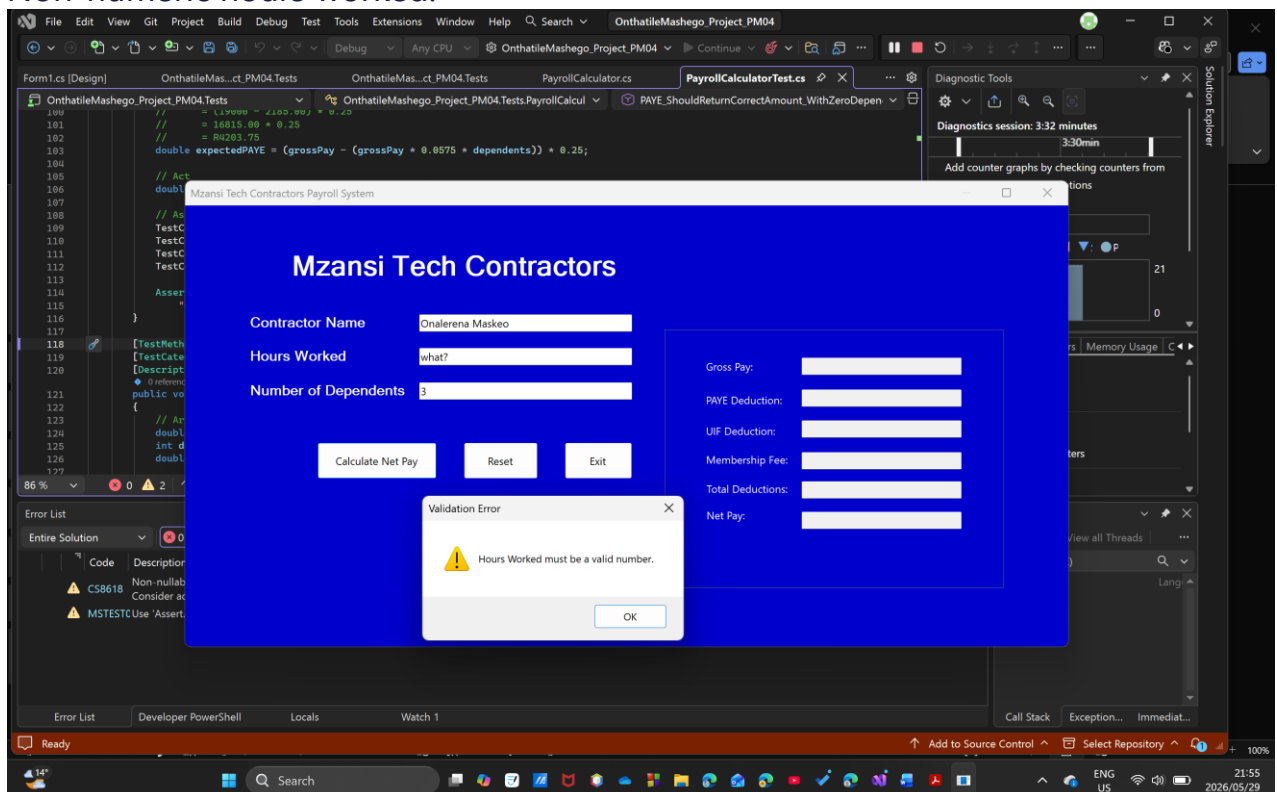
Blank Name:



Negative hours worked:



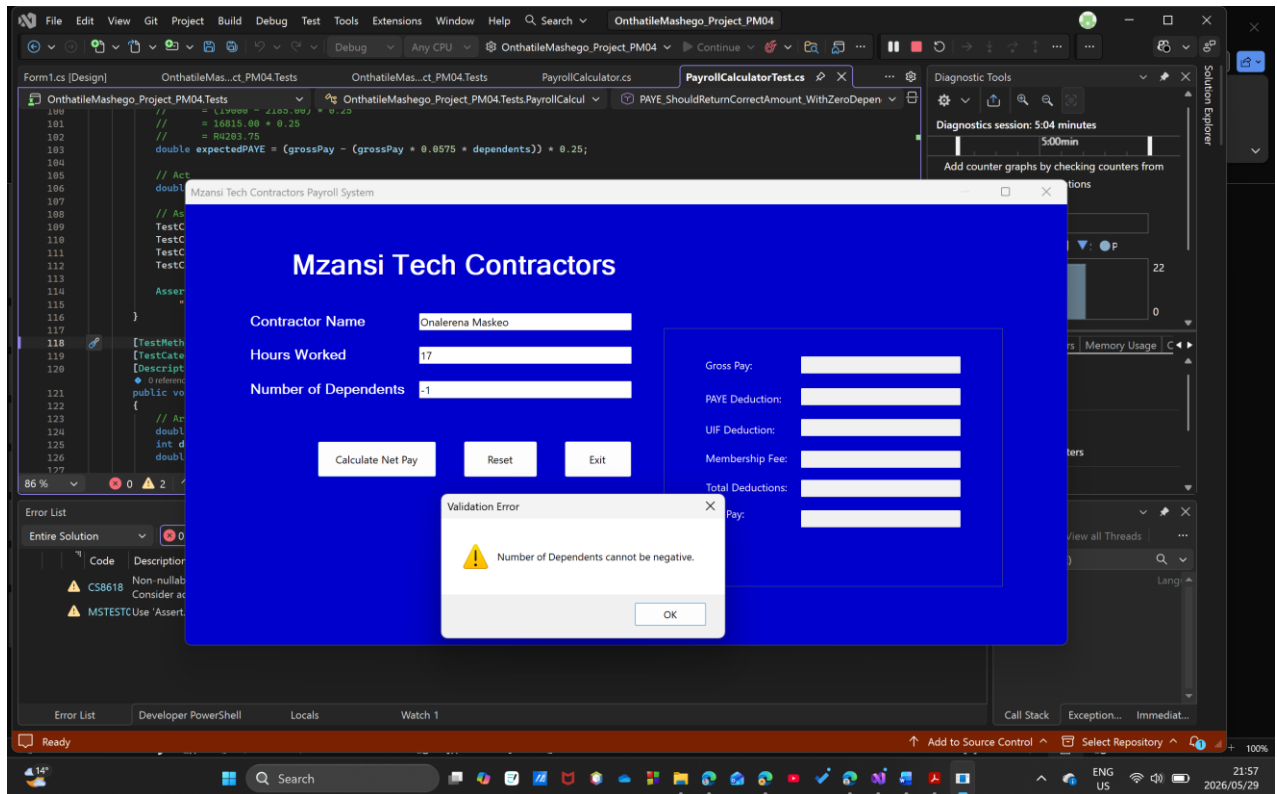
Non-numeric hours worked:



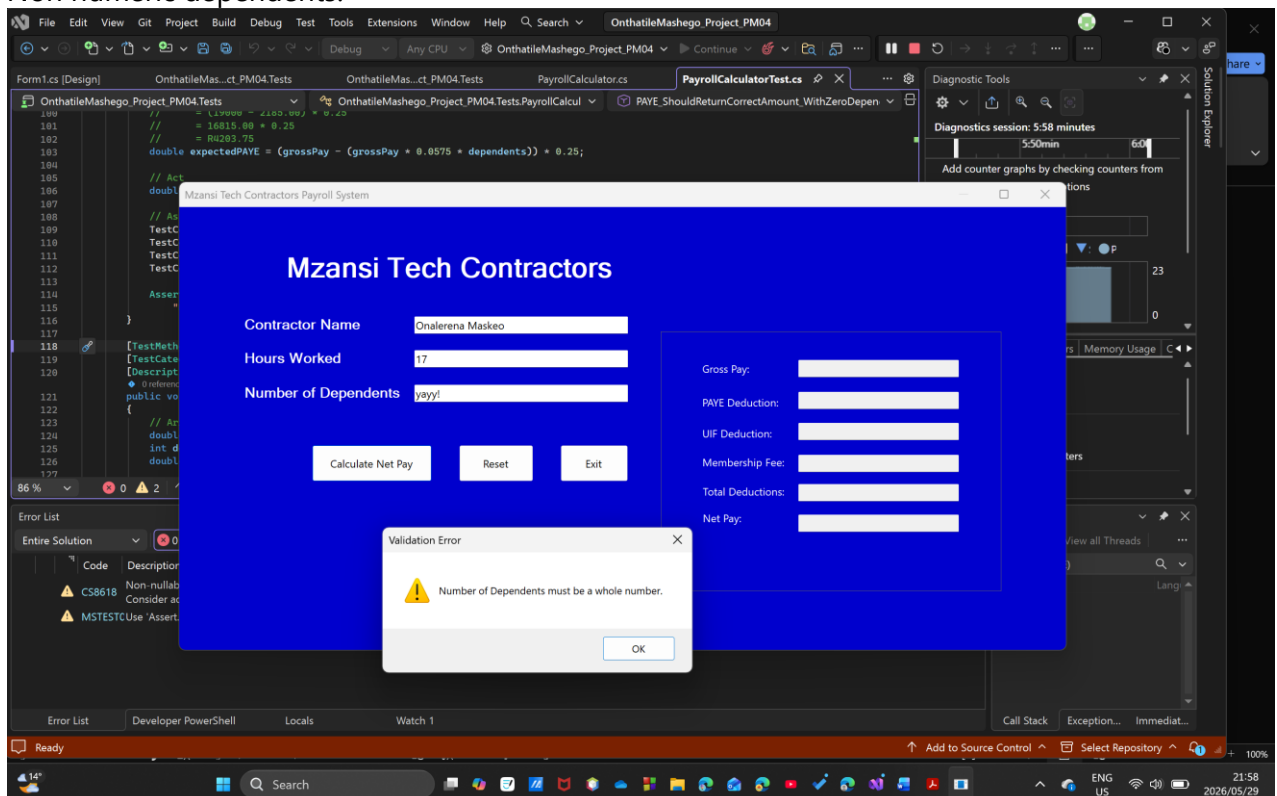
Negative Dependants:

26 | Page

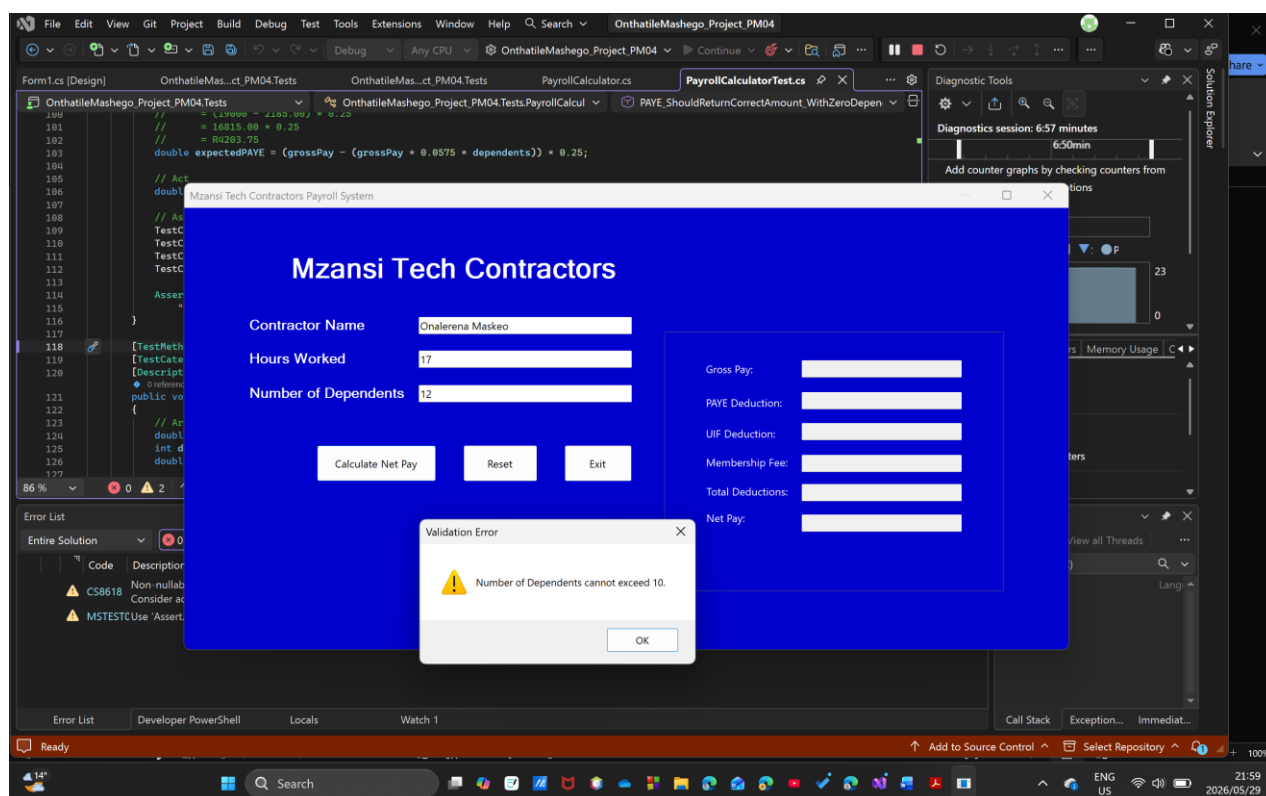
In line with industry requirements, CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004). **Directors: R Meeske, N Tattersall, E Ferreira, B Brown | Co. Reg No:2005/038944/07**



Non numeric dependents:



Dependents greater than 10 (>10):



Test Summary

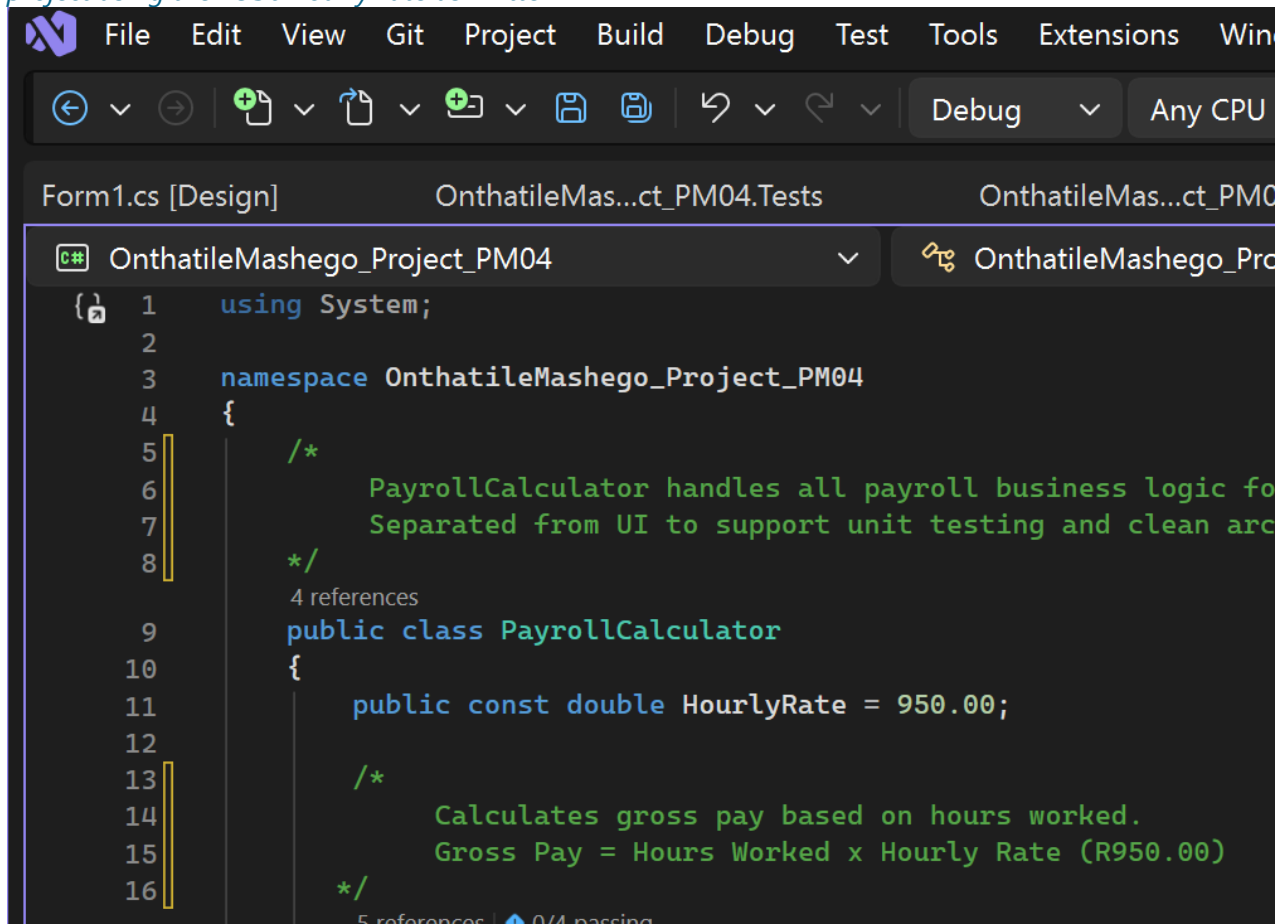
Test Level	Total Tests	Passed	Failed	Defects Found
Unit Testing (MSTest)	8	8	0	1 (D01)
Integration Testing	4	4	0	0
System Testing	5	5	0	1 (D02)
Validation Testing	6	6	0	0
Regression Testing (post-fix)	3	3	0	0

All defects identified during testing were corrected and retested. The full regression suite confirmed no regressions were introduced. The Mzansi Tech Contractors Payroll System is working correctly in accordance with all stated business rules and validation requirements.

END OF PAPER

Output Example

I would like to point out that the screenshot below was generated with an hourly pay rate of R150 which goes against what the Project has specified that the hourly rate should be R950. I have done the project using the R950 hourly rate as written

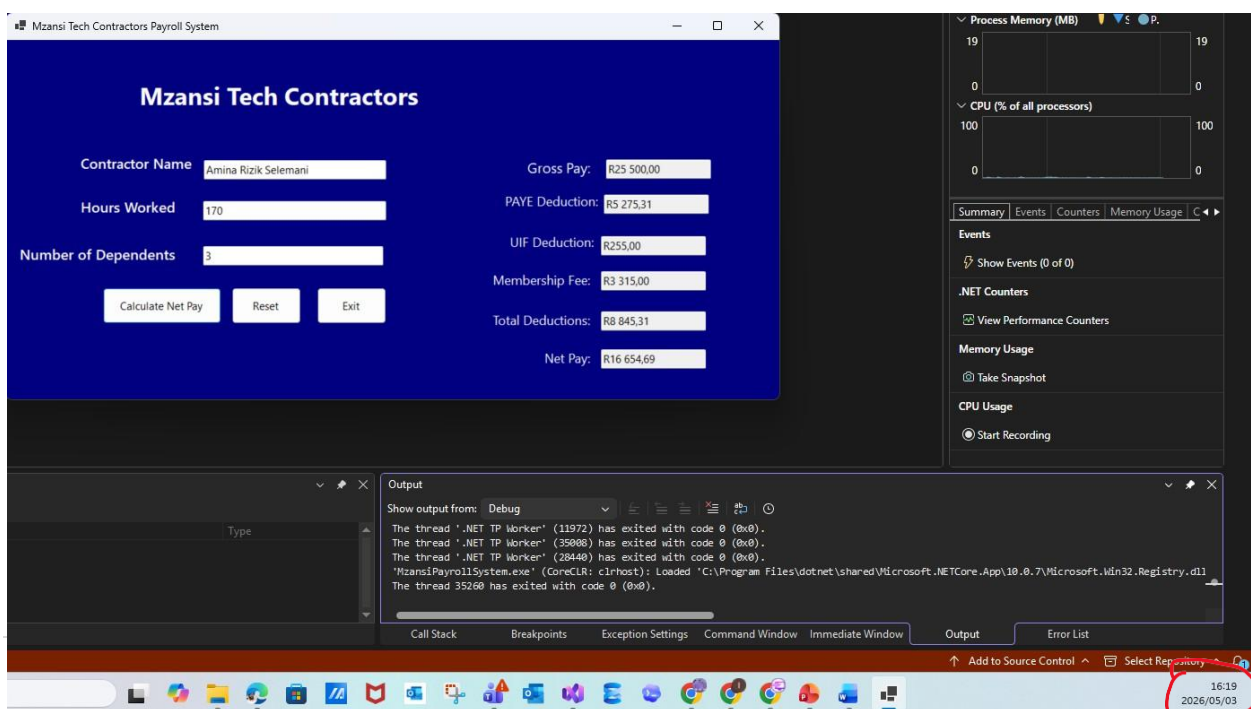


```

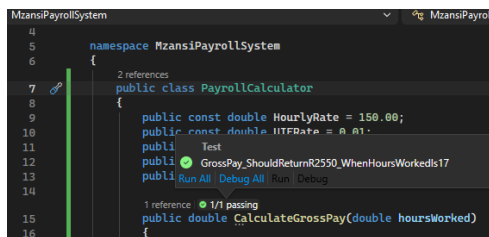
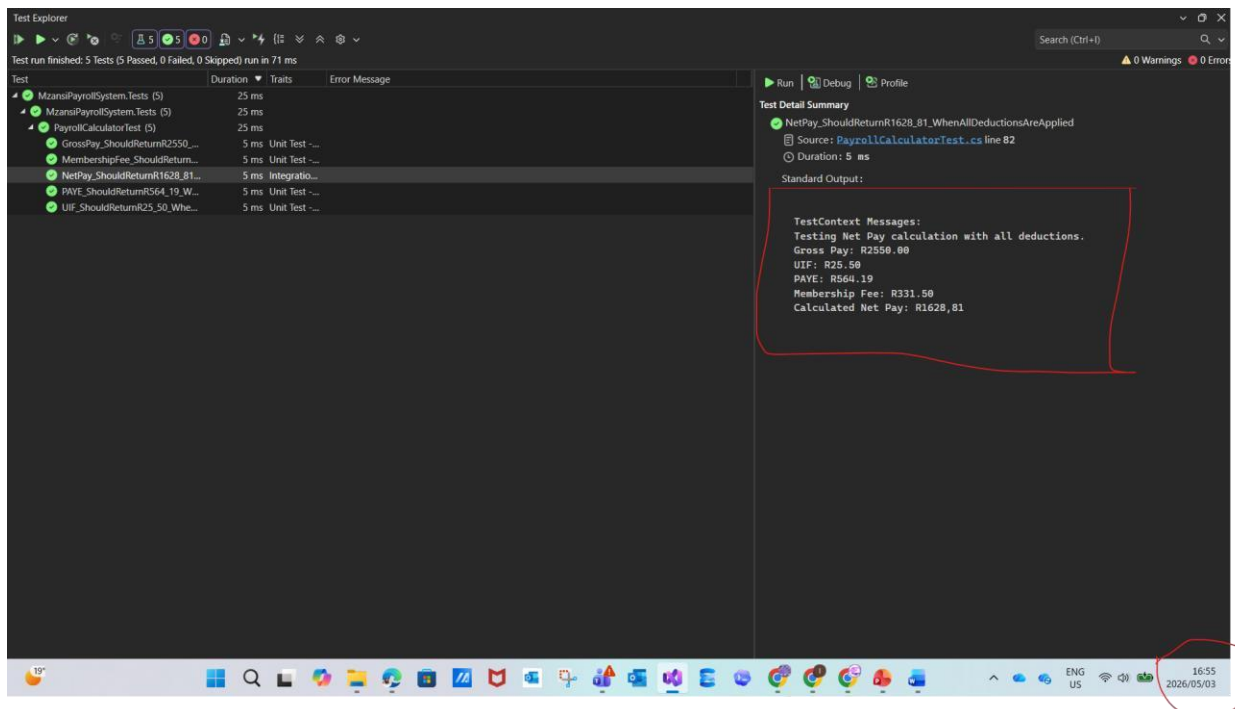
1  using System;
2
3  namespace OnthatileMashego_Project_PM04
4  {
5      /*
6       * PayrollCalculator handles all payroll business logic for
7       * Separated from UI to support unit testing and clean architecture
8       */
9      4 references
10     public class PayrollCalculator
11     {
12         public const double HourlyRate = 950.00;
13
14         /*
15          * Calculates gross pay based on hours worked.
16          * Gross Pay = Hours Worked x Hourly Rate (R950.00)
17          */
18     }
19     5 references | 0/4 passing

```

Provided screenshot example:



After test passed:



Github Submission (My Repository Link):

It says SDT612 but it is actually for SDT621, I had already created the repo.

[Onthatile-CTU/OnthatileMashego Practical Module PM04 SDT612: My Submission for the Practical Module PM04. Software design and testing C#. For SDT621 @ CTU](#)

<https://github.com/Onthatile-CTU/OnthatileMashego Practical Module PM04 SDT612/>

PM-04 Marking Rubric

Section	Criteria	Marks
User Interface Design (10)	Clear and professional layout	2
	Correct labels and input fields	2
	Proper alignment and spacing	2
	Appropriate use of controls	2
	Output section clearly structured	2
Input Validation (10)	Validates empty fields	2
	Validates numeric input	2
	Prevents negative values	2
	Validates dependents range (0–10)	2
	Displays user-friendly error messages	2
Payroll Calculations (20)	Correct Gross Pay calculation	4
	Correct UIF calculation (1%)	4
	Correct Membership Fee (13%)	4
	Correct PAYE calculation	4
	Correct Net Pay calculation	4
Application Functionality (10)	Calculate button works	3
	Reset button clears fields	3
	Exit button works	2
	System runs without errors	2
Unit Testing (20)	Test project created correctly	3
	PayrollCalculator class used	3
	Gross Pay test	3
	UIF test	3
	Membership Fee test	3
	PAYE test	3
	Net Pay test	2
Test Quality & Documentation (15)	Meaningful test names	3
	Use of TestCategory	3
	Use of TestContext logging	3
	Expected vs actual validation	3

	Professional structure	3
Integration & System Testing (10)	Integration testing explained	3
	System testing scenario	3
	End-to-end workflow tested	2
	Evidence provided (screenshots/logs)	2
Code Quality (5)	Clean, readable code	2
	Proper naming conventions	2
	Separation of logic (Calculator class)	1
TOTAL		100

PM-04 Mapping: Mzansi Tech Contractors Payroll System

PM-04 Requirement / Outcome	Project Activity	Evidence Required	Marks
Apply correct tests	Unit testing using MSTest for gross pay, UIF, PAYE, membership fee, and net pay	Test Explorer screenshot showing passed tests	15
Test system calculations	Verify all payroll formulas produce correct results	Test cases with expected vs actual results	15
Test integrated components	Confirm inputs, calculation class, and output textboxes work together	Screenshot of completed payroll form	10
Perform system testing	Test the full application from start to finish	System test table and screenshots	10
Identify defects	Record any errors found during testing, such as wrong formula or failed reset button	Defect log / bug report	10
Debug source code	Correct identified errors in the application	Before-and-after explanation or screenshots	10
Retest fixed defects	Run the same test again after fixing the issue	Retest evidence showing pass result	10
Produce test report	Document test cases, expected results, actual results, and pass/fail status	Completed test report	10
Follow coding standards	Use clear names, readable code, and separated PayrollCalculator class	Source code screenshots / GitHub link	5
Provide complete project evidence	Submit application, tests, screenshots, and report	Final zipped project / GitHub repository	5
Total			100